BRNA Prototype - Detailed Functional Analysis

Complete Explanation of Every Function in the Program

PART 1: CORE DATA STRUCTURES (The Foundation)

1.1 Enum Classes - Defining the Building Blocks

```python
class NodeType(Enum):
    ROUTER = "router"
    SWITCH = "switch"
    PC = "pc"

    QNODE = "qnode"
```

What it does: Defines the four possible types of network devices in the BRNA architecture.

How it relates to BRNA theory:

- ROUTER = Neural Repeaters (microtubule-based quantum processors)
- SWITCH = Mycelial mesh fabric nodes (biological switching points)
- PC = End-user devices (interfaces to the biological network)
- QNODE = Pure quantum nodes (entanglement endpoints)

In the topology:

- Core zone uses ROUTER type (NR-A, B, C, D)
- Bio zone uses SWITCH (SW-BIO) and PC types
- HITL zone uses PC and SWITCH types
- Entanglement zone uses QNODE type

```python
class ZoneType(Enum):
    CORE = "core"
    BIO = "bio"
    HITL = "hitl"
```

```
    ENTANGLE = "entangle"
```

What it does: Categorizes nodes into four functional zones.

BRNA Architecture Reference:
From the 5-layer model, zones correspond to:

- CORE = Layers 1-3 (Bio-Physical, Sub-Network, Routing)
- BIO = Layer 2 (Mycelial Mesh) + Storage
- HITL = Layer 5 (Human governance)
- ENTANGLE = Layer 3 (Quantum routing)

```python
class ProtocolType(Enum):
    QSP = "QSP"                     # Quantum Sensory Protocol
    MYCO_RIP = "MYCO-RIP"           # Mycelial Routing
    DNA_STORE = "DNA-STORE"         # DNA Storage
    HITL_CTRL = "HITL-CTRL"         # Human Control
    ICMP = "ICMP"                    # Ping
    PPP = "PPP"                      # Paradox Prevention
    SPIN_BRIDGE = "SPIN-BRIDGE"      # Quantum-Digital Bridge
    BLAP = "BLAP"                    # Bio-Link Access
    HTTP = "HTTP"                     # Web
    SSH = "SSH"                       # Secure Shell
    SNMP = "SNMP"                     # Network Monitoring
    SYSLOG = "SYSLOG"                 # Logging
```

Protocol Theory Reference:

Protocol	BRNA Layer	Function in Architecture
QSP	L5 - Galactic Application	Intent-based messaging using phase sync
MYCO-RIP	L3 - Network	Fungal growth routing algorithm
DNA-STORE	L1 - Physical	Molecular encoding with Reed-Solomon
HITL-CTRL	L5 - Application	Human override with 2-of-2 consensus

PPP	L5 - Security	Prevents causality violations
SPIN-BRIDGE	L4 - Bridge	Converts qubits to classical bits
BLAP	L2 - Data Link	Mitochondrial DNA addressing

1.2 QuantumState Class - The Heart of Orch OR

```python
@dataclass
class QuantumState:
    coherence: float           # 0-100%
    spin_state: str            # ↑↓, ↑↑, ↓↓, superposition
    microtubule_class: str     # A, B, C, S
    frequency: float           # Hz
    decoherence_rate: float    # ns
    bell_state: Optional[str] = None  # |Φ+⟩, |Ψ-⟩

    entanglement_partner: Optional[str] = None
```

What it does: Models the quantum state of a biological neural repeater.

The Orch OR Connection:
From the Penrose-Hameroff theory:

```text
E_G = ħ/t
Where:
- E_G = gravitational self-energy
- ħ = reduced Planck constant

- t = time until collapse
```

In the code:

- `coherence` = How long the quantum state remains stable (related to 1/t)
- `decoherence_rate` = How fast the state decays (environmental interaction)
- `spin_state` = The actual quantum information (↑↓ represents superposition)

Microtubule Classes from BRNA spec:

- Class A: Standard neural repeater (NR-A, NR-B)
- Class B: Edge node with lower coherence (NR-C)
- Class C: Bio zone processor (NR-F)

- Class S: Superior hub node (NR-D)
- Class Q: Pure quantum node (QNodes)

Bell States - Quantum Entanglement:

- `|Φ+⟩` = (|00⟩ + |11⟩)/√2 (used for QNode-A/B)
- `|Ψ-⟩` = (|01⟩ - |10⟩)/√2 (used for QNode-C/D)

CHSH value >2.7 proves entanglement (classical max is 2.0)

1.3 BioMetrics Class - Living Network Parameters

```python
@dataclass
class BioMetrics:
    metabolic_rate: float      # ATP consumption
    temperature: float         # Celsius
    ph_level: float
    nutrient_level: float      # 0-100%
    neural_noise: float        # dB

    self_healing: bool = True
```

What it does: Models the biological requirements of living network nodes.

BRNA Biological Requirements (from spec):

```text
Metabolic Stability: ATP supply must be maintained
Thermal Window: 36°C - 39°C (body temperature range)
pH Balance: 7.2-7.4 optimal for cellular function

Self-Healing: Enabled on mycelial nodes (<50ms recovery)
```

Why these matter:

- Metabolic rate = Energy consumption (affects node availability)
- Temperature = Affects quantum coherence (too hot = decoherence)
- Nutrients = Fuel for biological processing (low = standby mode)
- Neural noise = Signal interference (affects fidelity)

1.4 NetworkNode Class - Complete Node Model

```python
@dataclass
```

```python
class NetworkNode:
    node_id: str
    node_type: NodeType
    zone: ZoneType
    subnet: str                 # IP network (10.0.x.x)
    role: str                    # Functional description
    ios_version: str             # OS version (Cisco/BRNA-OS)
    status: str                  # Active/Standby/Idle/Error
    quantum: Optional[QuantumState]
    bio: Optional[BioMetrics]
    interfaces: List[Dict]       # Network interfaces
    metrics: Dict[str, Any]      # Performance data
    protocols: List[ProtocolType]
    python_file: str             # Simulation module
    x: float                      # Canvas position
    y: float
```

What it does: The complete data model for every node in the network.

Mapping to BRNA Architecture:

Field	BRNA Concept	Example (NR-D)
node_id	Node identifier	"NR-D"
zone	Network zone	CORE
subnet	IP addressing	10.0.1.4/24
role	Functional role	"Hub Router — Central Resonance Node"
quantum	Orch OR state	coherence=97%, spin="↑↓ Superposition"
bio	Biological metrics	temp=36.9°C, nutrients=98%
protocols	Stack layers	QSP, MYCO-RIP, DNA-STORE, HITL-CTRL

x,y	Topology position	(450, 220)

1.5 NetworkLink Class - Connections Between Nodes

```python
@dataclass
class NetworkLink:
    from_node: str
    to_node: str
    link_type: str      # serial, eth, quantum, swap
    label: str
    bandwidth: float = 1000     # Mbps
    latency: float = 1.0        # ms
    resonance_cost: float = 1.0 # C_R metric
    active: bool = True
    traffic: float = 0          # Animation

    highlight: bool = False
```

What it does: Models the connections in the network topology.

Link Types in BRNA:

Type	Physical Analogy	BRNA Concept
eth	Ethernet cable	Standard network link (1 Gbps)
serial	Fiber optic	Mycelial strand (self-healing)
quantum	Entangled particles	Zero-latency quantum channel
swap	Quantum repeater	Entanglement swapping link

Resonance Cost (C_R):

```text
C_R = sum(1/coherence_time + metabolic_overhead)
```

Used in modified Dijkstra routing (Scenario 10)

PART 2: TOPOLOGY GENERATION - The Physical Network

2.1 create_topology_with_positions() - Building the 16-Node Network

```python
def create_topology_with_positions() -> Dict[str, NetworkNode]:
    """Create the default topology with canvas positions"""

    nodes = {}
```

What it does: Instantiates all 16 nodes with their complete configurations.

The 4-Zone Architecture (from BRNA spec):

```text
┌─────────────────────────────────────────────────┐
│ CORE ZONE (10.0.1.0/24)                         │
│  NR-A(94%) ─ NR-B(91%) ─ NR-C(88%)              │
│       └──────── NR-D(97%) ────────┘              │
│         Hub with 5 interfaces                    │
├─────────────────────────────────────────────────┤
│ BIO ZONE (10.0.2.0/24)                          │
│  SW-BIO (Mycelial Mesh)                         │
│   ├─ NR-E (89%)                                  │
│   ├─ DNA-Store (10^18 bytes/cc)                  │
│   ├─ NR-F (87%)                                  │
│   ├─ PC-Bio1 (Monitor)                           │
│   └─ PC-Bio2 (Test)                              │
├─────────────────────────────────────────────────┤
│ HITL ZONE (10.0.3.0/24)                         │
│  SW-HITL (Governance)                           │
│   ├─ HITL-GW (Override Gateway)                  │
│   ├─ Monitor-1 (Fidelity)                        │
│   ├─ Monitor-2 (Ethics)                          │
│   ├─ PC-HITL1 (Operator 1)                       │
│   └─ PC-HITL2 (Operator 2)                       │
├─────────────────────────────────────────────────┤
│ ENTANGLEMENT ZONE (10.0.4.0/24)                 │
│  QNode-A(|Φ+⟩) ─ QNode-B(|Φ+⟩) ─ QNode-C(|Ψ-⟩) ─ QNode-D(|Ψ-⟩)
│        99%          99%          98%          98%   │
└─────────────────────────────────────────────────┘
```

Key Node Configurations:

```python
# Core Hub - NR-D (Central Router)
nodes["NR-D"] = NetworkNode(
    node_id="NR-D",
    node_type=NodeType.ROUTER,
    zone=ZoneType.CORE,
    subnet="10.0.1.4/24",
    role="Hub Router — Central Resonance Node",
    quantum=QuantumState(
        coherence=97.0,           # Highest in network
        spin_state="↑↓ Superposition",
        microtubule_class="S",     # Special hub class
        frequency=42.0,            # Optimal frequency
        decoherence_rate=1.2       # Slowest decay
    ),
    bio=BioMetrics(
        metabolic_rate=0.75,       # Efficient
        temperature=36.9,           # Optimal
        nutrient_level=98.0,        # Well-fed
        neural_noise=0.01           # Clean signal
    ),
    interfaces=[                    # 5 interfaces
        {"name": "Fa0/0", "ip": "10.0.1.4"},
        {"name": "Fa1/0", "ip": "10.0.2.254"},  # To Bio
        {"name": "Fa1/1", "ip": "10.0.3.254"},  # To HITL
    ],
    protocols=[ProtocolType.QSP, ProtocolType.MYCO_RIP,
               ProtocolType.DNA_STORE, ProtocolType.HITL_CTRL],
    x=450, y=220

)
```

Why this matters: NR-D is the central hub - all cross-zone traffic routes through it. Its high coherence (97%) ensures reliable quantum operations.

```python
# DNA Storage Node
nodes["DNA-Store"] = NetworkNode(
    node_id="DNA-Store",
    node_type=NodeType.PC,
    zone=ZoneType.BIO,
    subnet="10.0.2.2",
    role="PC — DNA Data Lake",
    metrics={
        "write_latency": 800,         # ms
        "throughput": 2100,            # MB/s
        "error_rate": 1e-8,            # 1 in 100 million
        "storage_used": 34,             # Percent
        "density": "10^18 bytes/cc"     # Theoretical max
    },
    protocols=[ProtocolType.DNA_STORE],
    x=250, y=480
```

```
)
```

DNA Storage Theory:
From BRNA spec: "10^18 bytes per cubic millimeter, 215 Petabytes per gram"
The code models this with `density: "10^18 bytes/cc"` in metrics.

2.2 create_enhanced_links() - Defining Connections

```python
def create_enhanced_links() -> List[NetworkLink]:
    return [
        # Core mesh (serial = mycelial)
        NetworkLink("NR-A", "NR-B", "serial", "Serial", latency=2.5),
        NetworkLink("NR-B", "NR-C", "serial", "Serial", latency=2.7),

        # Core to hub (ethernet)
        NetworkLink("NR-A", "NR-D", "eth", "Fa0/1", latency=1.0),
        NetworkLink("NR-B", "NR-D", "eth", "Fa1/0", latency=1.1),
        NetworkLink("NR-C", "NR-D", "eth", "Fa1/0", latency=1.2),

        # Hub to zones
        NetworkLink("NR-D", "SW-BIO", "eth", "Fa1/0", latency=1.5),
        NetworkLink("NR-D", "SW-HITL", "eth", "Fa1/1", latency=1.5),

        # Bio mesh (mycelial fabric)
        NetworkLink("SW-BIO", "DNA-Store", "eth", "Gi0/2", latency=2.0),
        NetworkLink("SW-BIO", "NR-E", "eth", "Gi0/1", latency=2.0),
        NetworkLink("SW-BIO", "NR-F", "eth", "Gi0/3", latency=2.1),

        # Quantum entanglement
        NetworkLink("QNode-A", "QNode-B", "quantum", "QSP pair 1",
latency=0.0),
        NetworkLink("QNode-C", "QNode-D", "quantum", "QSP pair 2",
latency=0.0),
        NetworkLink("QNode-B", "QNode-C", "swap", "Swap", latency=0.5),

    ]
```

Link Theory:

Serial Links (Mycelial):

- Based on Physarum polycephalum (slime mold) growth patterns
- Self-healing capability (<50ms reroute)
- Higher latency (2.5-2.7ms) but more resilient

Quantum Links:

- Zero latency (0.0ms) - theoretical instantaneous communication
- Based on Bell state entanglement
- `latency=0.0` represents non-local correlation

Swap Links:

- Enables entanglement swapping (A-B + B-C → A-C)
- `latency=0.5ms` accounts for Bell state measurement time
- From BRNA spec: "Extends network range without classical signal degradation"

PART 3: VISUALIZATION ENGINE - Bringing Theory to Life

3.1 HTMLVisualizer Class - The Browser Interface

```python
class HTMLVisualizer:
    def __init__(self, nodes: Dict[str, NetworkNode], links:
List[NetworkLink]):
        self.nodes = nodes
        self.links = links

        self.temp_file = None
```

What it does: Converts the Python data model into an interactive HTML5 canvas visualization.

How it works:

```text
┌──────────────────────────────────────────────────────────┐
│  Python Objects                  HTML/JavaScript          │
│  ┌──────────────┐                ┌──────────────┐         │
│  │ NetworkNode │──JSON──────→ │ const nodes │         │
│  │  - id:NR-A  │  serialize     │   [array]    │         │
│  │  - x:200    │                │              │         │
│  │  - y:120    │                │              │         │
│  └──────────────┘                └──────────────┘         │
│                                        │                  │
│                                        ▼                  │
│                                ┌──────────────┐          │
│                                │ HTML5 Canvas │          │
```

```
│                                │ drawZones()      │         │
│                                │ drawLinks()      │         │
│                                │ drawNodes()      │         │
│                                │ drawPackets()    │         │
│                                └──────────────────┘         │
└─────────────────────────────────────────────────────────────┘
```

3.2 generate_html() - Creating the Visualization

```python
def generate_html(self, title: str) -> str:
    # Convert nodes to JavaScript array
    nodes_js = []
    for node_id, node in self.nodes.items():
        coherence = node.quantum.coherence if node.quantum else 0
        nodes_js.append(f"""
            {{ id: '{node_id}', x: {node.x}, y: {node.y},
               zone: '{node.zone.value}', type: '{node.node_type.value}',
               coherence: {coherence},
               status: '{node.status}' }}""")
```

What it does: Creates a complete HTML file with embedded JavaScript.

The Canvas Drawing Functions:

```javascript
function drawZones() {
    // Draw zone backgrounds with dashed borders
    drawZone(100, 50, 850, 270, 'core');      // Blue
    drawZone(50, 300, 450, 590, 'bio');       // Purple
    drawZone(500, 300, 900, 590, 'hitl');     // Amber
    drawZone(100, 600, 850, 670, 'entangle'); // Green
}
```

Zone Theory: Each zone has a specific function in BRNA:

- Core (Blue) - Quantum processing (L1-3)
- Bio (Purple) - DNA storage + mycelial mesh (L2)
- HITL (Amber) - Human governance (L5)
- Entangle (Green) - Zero-latency links (L3)

```javascript
function drawNodes() {
    nodes.forEach(node => {
        // Different shapes for different node types
```

```
        switch(node.type) {
            case 'router': // Diamond for neural repeaters
                // Matches microtubule structure visualization
                break;
            case 'qnode':  // Circle for quantum nodes
                // Inner circle represents Bell state
                ctx.arc(node.x, node.y, 8, 0, Math.PI * 2);
                break;
        }

        // Coherence indicator (color-coded)
        if (node.coherence > 90) color = '#2d7a4f';  // Good
        else if (node.coherence > 85) color = '#f59e0b'; // Warning
        else color = '#c0392b';  // Critical
    });

}
```

Visual Feedback Theory:

- Pulse effect = Active quantum processing (Orch OR events)
- Moving packets = Data flow through the network
- Color-coded coherence = Real-time quantum health monitoring

```javascript
function drawPackets() {
    packets.forEach(packet => {
        packet.progress += 0.02;  // Animate along link
        // Draw glowing sphere
        ctx.shadowColor = packet.color;
        ctx.shadowBlur = 15;
        ctx.arc(x, y, 6, 0, Math.PI * 2);
    });

}
```

Packet Animation Theory: Represents quantum information transfer. Different colors represent different protocols:

- Blue (#1a6faf) = QSP packets
- Purple (#6b3fa0) = DNA-STORE packets
- Green (#2d7a4f) = Quantum entanglement sync

PART 4: SIMULATION ENGINE - Testing the Theory

4.1 EnhancedBRNASimulator Class

```python
class EnhancedBRNASimulator:
    def __init__(self):
        self.nodes = {}
        self.links = []
        self.test_results = []
        self.log = []
        self.rng = random.Random(42)  # Seeded for reproducibility

        self.current_step = 0
```

What it does: Runs test scenarios and collects results.

Why seed 42? Ensures reproducible simulations - same random sequence every run.

4.2 run_scenario() - Executing Tests

```python
def run_scenario(self, scenario: TestScenario) -> TestResult:
    self.load_scenario(scenario)

    for step in scenario.test_steps:
        self.current_step = step['step']
        result = self._execute_step(step)
        step_results.append(result)

        if result["status"] == "FAILED" and step.get("critical", False):
            break  # Critical failure - abort
```

What it does: Steps through each test, collecting results and metrics.

4.3 _execute_step() - Individual Test Steps

```python
def _execute_step(self, step: Dict) -> Dict:
    # Simulate execution time
    time.sleep(0.5)

    # Different success probabilities based on test type
    success_prob = 0.95  # Default 95% success

    if "decoherence" in step['name'].lower():
```

```
        success_prob = 0.90  # Decoherence harder to control
    elif "corruption" in step['name'].lower():
        success_prob = 0.98  # ECC very reliable
    elif "swap" in step['name'].lower():
        success_prob = 0.94  # Entanglement swapping

    # Generate result
    if self.rng.random() < success_prob:
        status = "PASSED"
    else:
        status = "FAILED" if self.rng.random() < 0.5 else "WARNING"
```

Theory in Practice:

- Decoherence (90%) - Quantum states naturally decay (Orch OR theory)
- Corruption recovery (98%) - Reed-Solomon ECC very effective
- Swap success (94%) - Entanglement swapping has some failure rate

4.4 _collect_metrics() - Network Statistics

```python
def _collect_metrics(self) -> Dict:
    metrics = {
        "avg_coherence": 0,
        "total_nodes": len(self.nodes),
        "active_nodes": 0,
        "entanglement_pairs": 0
    }

    for node in self.nodes.values():
        if node.status == "Active":
            metrics["active_nodes"] += 1
        if node.quantum and node.quantum.bell_state:
            metrics["entanglement_pairs"] += 1

    # Calculate averages
    if coherence_count > 0:
        metrics["avg_coherence"] = coherence_sum / coherence_count
```

What it measures:

- Average coherence - Overall quantum health of network
- Active nodes - Network availability
- Entanglement pairs - Number of quantum links established

PART 5: TEST SCENARIOS - Validating the Architecture

5.1 Scenario 1: Orch OR Coherence Test

```python
def get_scenario_orch_or_coherence():
    test_steps = [
        {
            "step": 1,
            "name": "Initialize microtubule bundles",
            "target": "NR-A",
            "expected": "Coherence > 90%"
        },
        {
            "step": 2,
            "name": "Apply Orch OR trigger",
            "command": "apply_orch_trigger(threshold=0.95)",
            "expected": "OR collapse probability < 5%"
        }

    ]
```

What it tests: The core Orch OR theory - can microtubules maintain quantum coherence?

The Theory:
From Penrose-Hameroff: Quantum states in microtubules collapse at specific thresholds (Orchestrated Objective Reduction). This test verifies that collapse probability stays below 5% under normal load.

Success Criteria:

- Initial coherence >90%
- Collapse probability <5%
- Decoherence rate <3.0ns
- Stress test keeps coherence drop <15%

5.2 Scenario 2: DNA Store Recovery Test

```python
def get_scenario_dna_store_recovery():
    test_steps = [
        {
```

```
            "step": 2,
            "name": "Apply Reed-Solomon encoding",
            "command": "rs_encode(symbols=(255,223))",
            "expected": "Redundancy 14%"
        },
        {
            "step": 3,
            "name": "Simulate sequence corruption",
            "command": "corrupt_dna(rate=0.10)",
            "expected": "10% errors introduced"
        },
        {
            "step": 4,
            "name": "Recover data with ECC",
            "command": "rs_decode()",
            "expected": "100% recovery"
        }

    ]
```

What it tests: DNA storage reliability with error correction.

The Theory:
Reed-Solomon (255,223) means:

- 255 total symbols
- 223 data symbols
- 32 error correction symbols
- Can correct up to 16 symbol errors (about 6.3%)

The test introduces 10% corruption - more than theoretical max - but with 14% redundancy, recovery should still be 100%.

5.3 Scenario 3: Entanglement Swapping Test

```python
def get_scenario_entanglement_swapping():
    test_steps = [
        {
            "step": 1,
            "name": "Create Bell pair A-B",
            "command": "create_bell_pair(state='|Φ+⟩')",
            "expected": "Fidelity > 98%"
        },
        {
            "step": 3,
            "name": "Perform entanglement swap B-C",
            "expected": "Swap success > 90%"
        },
```

```
        {
            "step": 4,
            "name": "Verify A-D entanglement",
            "command": "bell_test(node_a, node_d)",
            "expected": "CHSH > 2.7"
        }

    ]
```

What it tests: Can we create quantum links between non-adjacent nodes?

The Quantum Theory:

```text
Initial state:
A──(entangled)──B    C──(entangled)──D

After swap at B-C:
A──(entangled)──────────────────D

CHSH test proves entanglement:
Classical limit: S ≤ 2
Quantum limit: S ≤ 2√2 ≈ 2.828

BRNA achieves: 2.82 ✓
```

5.4 Scenario 4: HITL Override Test

```python
def get_scenario_hitl_override():
    test_steps = [
        {
            "step": 1,
            "name": "Generate critical packet",
            "command": "send_critical(priority='CRITICAL',
orch_collapse=0.98)",
            "expected": "Ethics engine flags packet"
        },
        {
            "step": 2,
            "name": "Queue for HITL review",
            "target": "HITL-GW",
            "expected": "Packet queued, operator notified"
        },
        {
            "step": 3,
            "name": "Simulate operator approval",
            "command": "operator_override(decision='APPROVE')",
            "expected": "Override recorded"
```

```
        }

    ]
```

What it tests: Human governance over critical quantum decisions.

The Ethics Theory:
From BRNA spec: "PPP (Paradox Prevention Protocol) drops causality-violating packets"

- Critical packets (orch_collapse >95%) require human approval
- 2-of-2 consensus policy
- 30-second timeout

5.5 Scenario 5: Mycelial Self-Healing Test

```python
def get_scenario_mycelial_self_heal():
    test_steps = [
        {
            "step": 2,
            "name": "Simulate link failure",
            "command": "fail_link(link='SW-BIO-NR-E')",
            "expected": "Link down, rerouting triggered"
        },
        {
            "step": 3,
            "name": "Measure healing time",
            "expected": "Healing < 50 ms"
        }

    ]
```

What it tests: Self-healing capability of mycelial mesh.

The Biological Theory:
Physarum polycephalum (slime mold) optimizes network paths:

- When a link fails, fungal hyphae grow new connections
- Rerouting in <50ms (theoretical minimum)
- Path redundancy: 3 alternative routes in Bio zone

5.6 Scenario 6: QSP Handshake Test

```python
```

```python
def get_scenario_qsp_handshake():
    test_steps = [
        {
            "step": 1,
            "name": "Send QSP SYN",
            "command": "qsp_send(flags='SYN', resonance_id='0x4A2F')"
        },
        {
            "step": 4,
            "name": "Verify phase sync",
            "command": "measure_phase_coherence()",
            "expected": "Phase difference < 0.1 rad"
        }

    ]
```

What it tests: Quantum Sensory Protocol connection establishment.

QSP vs TCP/IP:

TCP/IP	QSP
3-way handshake (SYN, SYN-ACK, ACK)	Phase synchronization
Packet delivery	State resonance
Binary logic	Intent-focused
Lossless	Coherence-based

QSP Frame Structure:

```text
[Resonance ID (16B) | Phase State (4B) | DNA Checksum (32B) | Payload]
```

5.7 Scenario 7: Paradox Prevention Test

```python
def get_scenario_paradox_prevention():
    test_steps = [
        {
            "step": 2,
            "name": "Generate causality violation",
```

```
            "command": "send_packet(timestamp='-50ms',
entropy_decrease=0.2)",
            "expected": "Packet flagged"
        },
        {
            "step": 3,
            "name": "Verify PPP drop",
            "expected": "Packet dropped, violation logged"
        }

    ]
```

What it tests: Prevention of time-travel paradoxes in quantum communication.

The Physics Theory:
If information travels faster than light (zero latency), it could create causality violations:

- Sending a message to your past self
- Decreasing local entropy (creating order) violates 2nd law

PPP checks: If packet would decrease entropy beyond threshold → DROP

5.8 Scenario 8: Metabolic Cooling Test

```python
def get_scenario_metabolic_cooling():
    test_steps = [
        {
            "step": 2,
            "name": "Simulate overheating",
            "command": "increase_temp(delta=3.5)",
            "expected": "Coherence drops"
        },
        {
            "step": 3,
            "name": "Activate metabolic cooling",
            "command": "metabolic_cooling(intensity=0.8)",
            "expected": "Temperature decreases"
        }

    ]
```

What it tests: Biological temperature regulation.

The Biological Requirement:

- Optimal: 36-39°C (body temperature)

- Above 39°C → Rapid decoherence
- Cooling rate: 0.5°C/s (metabolic processes)

5.9 Scenario 9: Galactic Routing Test

```python
def get_scenario_galactic_routing():
    # Add Proxima Centauri node
    scenario.nodes["Proxima-Centauri"] = NetworkNode(
        node_id="Proxima-Centauri",
        node_type=NodeType.QNODE,
        zone=ZoneType.ENTANGLE,
        role="Interstellar Relay — Proxima Centauri",
        quantum=QuantumState(coherence=92.0, bell_state="|Φ+⟩")

    )
```

What it tests: Zero-latency communication across interstellar distances.

The Implication:

- Distance to Proxima Centauri: 4.24 light years
- Classical communication: 4.24 years
- Quantum entanglement: 0 seconds

Success Criteria:

```python
expected_results = {
    "latency": 0.0,  # ms (instantaneous)
    "distance": "4.24 light years",
    "fidelity": 92.0  # % (some decoherence over chain)

}
```

5.10 Scenario 10: Resonance Cost Routing

```python
def get_scenario_resonance_cost_routing():
    test_steps = [
        {
            "step": 2,
            "name": "Run modified Dijkstra",
            "command": "dijkstra_resonance(source='NR-A',
target='DNA-Store')",
            "expected": "Path found with min C_R"
```

```
        }
    ]

    expected_results = {
        "resonance_cost_formula": "C_R = sum(1/coherence_time +
metabolic_overhead)",
        "optimal_path": "NR-A → NR-D → SW-BIO → DNA-Store",
        "total_cost": 4.6

    }
```

What it tests: Routing based on quantum coherence, not just distance.

Traditional Dijkstra: Shortest path (min hops)
Modified Dijkstra: Minimum resonance cost (C_R)

```text
C_R = Σ (1/coherence_time + metabolic_overhead)

Example path NR-A to DNA-Store:
NR-A(94%) → NR-D(97%) → SW-BIO → DNA-Store

    1.06    +   1.03    +   1.5    +   1.0    = 4.59
```

PART 6: TERMINAL UI - User Interface

6.1 EnhancedTerminalUI Class

```python
class EnhancedTerminalUI:
    def __init__(self):
        self.simulator = EnhancedBRNASimulator()
        # ANSI color codes for terminal
        self.RED = "\033[91m"
        self.GREEN = "\033[92m"

        self.BLUE = "\033[94m"
```

What it does: Provides text-based interface for running simulations.

6.2 print_menu() - Main Interface

```python
```

```python
def print_menu(self):
    print(f"  {self._c('1', self.GREEN)}. Run Pre-defined Test Scenario")
    print(f"  {self._c('2', self.GREEN)}. Launch Topology Visualizer Only")
    print(f"  {self._c('3', self.GREEN)}. Create Custom Scenario")
    print(f"  {self._c('4', self.GREEN)}. View Results")
    print(f"  {self._c('5', self.GREEN)}. Export Results")
    print(f"  {self._c('6', self.GREEN)}. Network Status")
```

Menu Options Explained:

1. Run Test - Executes one of 10 pre-defined scenarios with visualization
2. Visualizer Only - Just shows the topology without running tests
3. Custom Scenario - User builds own test sequence
4. View Results - Shows history of test runs
5. Export Results - Saves to JSON for documentation
6. Network Status - Displays current node states

6.3 network_status_menu() - Network Overview

```python
def network_status_menu(self):
    # Show zone distribution
    for zone, count in zone_counts.items():
        print(f"    {zone.upper()}: {count} nodes")

    # Show quantum stats
    if quantum_nodes > 0:
        print(f"    Average coherence: {avg_coherence:.1f}
```